



Service-oriented Application Composition with Evolutionary Heuristics and Multiple Criteria

HANGJUNG ZO, Kaist College of Business

DEREK L. NAZARETH, University of Wisconsin-Milwaukee

HEMANT K. JAIN, University of Tennessee Chattanooga

10

The need to create and deploy business application systems rapidly has sparked interest in using web services to compose them. When creating mission-critical business applications through web service compositions, in addition to ensuring that functional requirements are met, designers need to consider the end-to-end reliability, security, performance, and overall cost of the application. As the number of available coarse-grain business services grows, the problem of selecting appropriate services quickly becomes combinatorially explosive for realistic-sized business applications. This article develops a business-process-driven approach for composing service-oriented applications. We use a combination of weights to explore the entire QoS criteria landscape through the use of a multi-criteria genetic algorithm (GA) to identify a Pareto-optimal multidimensional frontier that permits managers to trade off conflicting objectives when selecting a set of services. We illustrate the effectiveness of the approach by applying it to a real-world drop-ship business application and compare its performance to another GA-based approach for service composition.

CCS Concepts: • **Information systems** → **Information systems applications**; **Decision support systems**; **World Wide Web**; **Web services**; • **Computing methodologies** → **Machine learning**; **Machine learning approaches**; *Bio-inspired approaches*; Genetic algorithms; • **Applied computing** → **Enterprise computing**; **Business process management**;

Additional Key Words and Phrases: Service-oriented applications, multiple criteria decision making

ACM Reference format:

Hangjung Zo, Derek L. Nazareth, and Hemant K. Jain. 2019. Service-oriented Application Composition with Evolutionary Heuristics and Multiple Criteria. *ACM Trans. Manage. Inf. Syst.* 10, 3, Article 10 (October 2019), 28 pages.

<https://doi.org/10.1145/3354288>

1 INTRODUCTION

ORGANIZATIONS are increasingly seeking agile and flexible application systems to respond quickly and effectively to dynamic pressures in competitive business environments. Firms recognize that meeting rapidly changing requirements is difficult to accomplish using traditional

Authors' addresses: H. Zo, School of Business and Technology Management, College of Business, KAIST, 291 Daehak-ro, Yuseong-gu, Daejeon, Republic of Korea, 34141; email: joezo@kaist.edu; D. L. Nazareth, Lubar School of Business, University of Wisconsin-Milwaukee, P.O. Box 742, Milwaukee, WI 53201; email: derek@uwm.edu; H. K. Jain, Gary W. Rollins College of Business, University of Tennessee Chattanooga, 615 McCallie Ave, Chattanooga, TN 37403; email: hemant-jain@utc.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2019 Association for Computing Machinery.

2158-656X/2019/10-ART10 \$15.00

<https://doi.org/10.1145/3354288>

development technologies. Application developers need a flexible and agile approach that can leverage rapidly evolving software technology and newer deployment mechanisms. Service-oriented architecture represents a promising option, as it allows business application systems to be composed from commercially available coarse-grain web services to support various business tasks. Additionally, as business conditions change, applications can be quickly realigned with business needs by replacing existing services with more appropriate ones. This approach provides increased flexibility, allowing IT to quickly respond to the changing needs of business (Choi et al. 2010). Web services represent software constructs that deliver specific business functionality using standard interfaces (Papazoglou and D. 2003). Recently, cloud computing providers have moved beyond the simple platform as a service (PaaS) and infrastructure as a service (IaaS) models and now routinely offer software as a service (SaaS) options to provide complete solutions to organizations that would prefer to invest their human capital in areas of their core competence. Providers such as ComponentSource.com, SAP, Google Cloud Platform, AppExchange, and IBM's Web Sphere Commerce Service offer hosted web services that support discrete business tasks that can be composed into an application (Clabby 2002).

With the availability of robust web services, organizations can quickly compose business applications that are agile and cost-effective. For example, an e-commerce application can be fashioned through the use of web services for order capture, order validation, inventory verification, shopping-cart management, and payment processing. Currently, there are several dozen web services that provide functionality for each of these tasks. The market for these products is expanding. This approach to building business applications provides organizations with the flexibility to select the service that meets their needs most effectively, verifying that the service is compatible with other services. Process orchestration and choreography address the interface issues and have been extensively addressed in the literature (Joachim 2011; Medjahed et al. 2003; ter Beek et al. 2007).

In this article, we focus on a business-process-driven methodology for composing applications. We utilize coarse-grain web services that can support specific tasks in the business process. Some services may support multiple tasks, and our methodology can accommodate that. The methodology does not utilize services that provide partial support for tasks. With an expanding market of available web services, the process of selecting the right services to support the business process becomes more challenging.

The structure of the article is as follows: We review the literature on service composition in Section 2. In Section 3, we develop robust business-oriented metrics for reliability, performance, security, and cost for individual web services. These metrics incorporate multiple hosting alternatives and ownership options. In Section 4, we develop methods for combining them to derive metrics for a complete end-to-end business process, given a variety of task combination patterns. In Section 5, we formulate the service selection problem and propose an innovative multi-criteria genetic algorithm solution approach to select the preferred combination of web services for the process. A real-world case study is employed to illustrate the effectiveness of the approach. The results are presented in Section 6, where we analyze its effectiveness using 1, 2, and 4 dimensions simultaneously. We also compare our approach to another GA-based approach, illustrating how our approach outperforms the other approach. Implications of this research, and its contributions, round out the article.

2 PRIOR RESEARCH ON SERVICE COMPOSITION

Service composition has been widely studied in the literature. A host of problems of different complexity have been studied, using a variety of different techniques. Given the overall complexity of the problem, most approaches tend to favor heuristic techniques utilizing evolutionary search strategies. This section provides an overview of the different approaches to service composition.

When selecting a service to support a task in a business process, the task's functional requirements must be met unequivocally. Selecting from the set of viable services requires the use of non-functional characteristics. Many of these non-functional aspects of services have been operationalized as quality of service (QoS) measures, including performance, reliability, security, availability, supportability, maintainability, data integrity, and cost, among others (Benlian et al. 2011; Cardoso et al. 2004; Grønmo and Jaeger 2005; OASIS 2004; Ran 2003; Hashmi et al. 2016; Delano et al. 2018). Performance, security, reliability, and cost are the most important and widely used objectives in service selection.

Assuming that the characteristic values of these QoS measures can be obtained from the provider of each service (increasingly available in service-level agreements), we need to compute the end-to-end values of reliability, performance, and security for the composed application. The end-to-end measure is a function of the business-process model that is specified by the patterns in which various tasks of the business process are executed. Moreover, these objectives for service selection do not work in concert. Thus, secure systems are likely to cost more and perform slower. Similarly, increased reliability will entail a greater cost and possibly lower security due to redundancy. It is imperative that system designers trade off these objectives when selecting services for an application. Given different organizational risk tolerances, environmental conditions, and competitive pressures, it is clear that different solutions will be preferred by different organizations. Additionally, most realistic business processes are composed of several dozen tasks, and with the increasing number of web services available for each task, the problem of selecting a set of services for supporting a business process that will satisfy the organization's objectives can become combinatorially explosive. In addition, some services can support multiple tasks in a process, introducing additional twists and making this more complex than traditional service selection. An exhaustive exploration of solution space is prohibitively expensive. A heuristic approach that can help managers make a trade-off between conflicting organizational objectives while selecting services is needed.

The web service selection problem has been acknowledged as an important issue in the adoption of web services, and several approaches have been developed to tackle the problem. Dai et al. (2014) propose an approach that uses availability and processing time, and develop a graph-based technique to select services for a small number of tasks and a limited set of candidate services. Ahmed et al. (2015) use response time as the basis for selection of functionally equivalent web services and employ a hidden Markov model to select "optimal" services. Huaizhou and Fan (2014) proposed an approach that seeks to minimize conflict among web services as part of the selection process. A relatively simplistic selection process is presented in Akingbesote et al. (2013), wherein a single web service is selected from a set of seven candidate web services using a weighted sum of normalized scores for response time, throughput, reliability, and adherence to best practices. Ramacher and Monch (2015) propose an approach that seeks to minimize total cost (including subscription and operation costs) subject to a performance time constraint, through the use of a hierarchical selection strategy using optimization. A different approach is proposed by Achbany et al. (2008), where service reputation is used to select web services for a set of 10 tasks, each of which can be provisioned by 3 web services. Their approach incorporates randomized reinforcement learning into the algorithm and employs a directed acyclic hypergraph representation, with 10K runs to incorporate the accumulated knowledge. The use of service cost and nine QoS measures was considered by Suleiman et al. (2011) to select a web service for a credit card processing task, choosing from a set of 22 candidate web services using weighted normalized metrics. Web service selection based on a bidding approach is proposed by He et al. (2014), wherein 6 web services are selected to support an automobile purchase decision, subject to response time and cost constraints.

QoS-aware web service composition has been studied in the extant literature (Rodriguez-Mier et al. 2017; Zou et al. 2014). There are several approaches to tackle this problem. The conventional approach is to treat QoS-aware web service composition as an optimization problem. Various search and optimization techniques including exhaustive search, integer programming, approximate algorithm, situation calculus, and so on have been applied to solve the problem (Zou et al. 2014). However, they cannot yield the globally optimal solution in most cases due to the extremely large solution space. One of the recent studies in this research stream presents a hybrid local-global search to generate semantic input-output based compositions with optimal end-to-end QoS, minimizing the number of services (Rodriguez-Mier, Mucientes and Lama 2017). Reducing the search space in multi-objective optimization is an effective way to tackle the problem, and a partial selection method based on the Pareto set model is proposed for efficient web service composition (Chen et al. 2015). Evolutionary algorithms have been extensively employed to solve multi-objective web service composition problems (Ramirez et al. 2017). An alternative approach is to employ artificial intelligence (AI) planning techniques (Carman et al. 2003; Hatzi et al. 2012). The service composition problem has also been tackled as a search tree, wherein 5 QoS objectives are considered on a synthetic process ranging up to 50 activities (Mabrouk et al. 2009). A model-driven approach to service composition was proposed, and decomposed into sequential steps of process modeling, service discovery, service selection, and service deployment (Grønmo and Jaeger 2005; Grønmo et al. 2004). However, no implementation is described. A transactional pattern approach was adopted in Bhiri et al. (2006), where they sought to maintain high reliability of services selected for a travel agent process. Serhani et al. (2005) proposed the use of a QoS broker-based architecture to assist in service selection. However, their research is aimed at selecting services with a view to maintaining acceptable levels of response time and availability in a marketplace. The problem can be converted to a planning problem in some cases, and solved using conventional AI planning algorithms. Zou et al. (2014) propose a new planning-based approach that automatically translates a QoS-aware web service composition to a planning problem with temporal and numerical features. They apply this approach to a dynamic web service composition situation instead of employing a predefined workflow model. Graph-based approaches represent another solution strategy for QoS-aware web service composition. In this approach, the entire composition is generated from relevant web services, and the global QoS of the composition is optimized. Recently, Jiang et al. (2014) generated the top k query compositions by relaxing the QoS optimality to enhance service availability. They generalize a new graph problem (top k directed acyclic graphs) that could be applied to model many other applications. In addition, the service providers search engine (SPSE) algorithm has been proposed for web service selection. It is a flexible and generic algorithm to support large-scale systems and reflect users' preferences on services (Zhao et al. 2012).

Several researchers have used genetic algorithms (GA) to address the service composition problem using QoS measures. Amiri and Serajzadeh (2010) employed a GA to identify candidate services based on user-specified constraints, using response time, execution cost, availability, reputation, and successful execution as QoS measures. They compared their results to a tabu search on the basis of execution time, noting superior performance for the GA. In subsequent research, they added a looping structure to the process structure (Amiri et al. 2013). The analysis is performed on the same set of tasks, and a comparison to a tabu search solution is reported. Tang and Ai (2010) employed three variants of GA and QoS measures of response time, cost, reputation, reliability, and availability to select web services. Using a normalized weighting combining function for a synthetic task, they reported computation time and overall fitness scores. Liu et al. (2010) used response time, execution cost, availability, and reputation as QoS measures to select services for processes that involved sequential tasks. A user-specified combination of weights was employed to create a hybrid GA-ACO (ant-colony optimization) solution. For a synthetic process involving 7

tasks, they reported that their approach outperformed a pure ACO solution. Liu and Weng (2012) utilized response time, cost, and reliability as QoS measures for service composition of processes involving sequence and parallel tasks. They examined cases involving user constraints and user-specified weights for normalized measures on a problem utilizing 6 tasks and 32 candidate web services, reporting the overall normalized scores. Su et al. (2007) used 5 unspecified QoS factors and a matrix representation of the process coupled with a ratio-based objective function to guide the GA. Using a synthetic process, they demonstrated that the modified GA outperforms traditional GA approaches. They also examined various mutation strategies and alternative objective functions. In a follow-up study, they tweaked the GA to include population diversity handling, once again comparing to the traditional GA, using the same problem and unspecified QoS measures (Ma and Zhang 2008). A GA-based approach using varying population sizes is presented in Mardukhi et al. (2013). It employs 6 QoS objectives and a normalized utility function on a real-world case of 6 tasks and 2,508 services to address service composition. The results are compared with a mixed integer programming strategy in terms of execution time and overall fitness. A GA approach modified to incorporate Q-learning was proposed in Elsayed et al. (2017). This study examined response time, cost, and reliability using a synthetic process. Comparisons were made with traditional GA and Q-learning, with the hybrid approach outperforming the others on a cumulative reward function. Another hybrid GA-ACO approach was utilized in Yang et al. (2010). Using synthetic tasks and cost and response time for QoS measures, they examine the effect of varying the user constraints on the performance of the hybrid approach, which outperforms traditional GA and ACO approaches. Shahrokh and Safi-Esfahani (2016) use a modified GA approach for the service selection problem and report that it outperforms traditional GA solutions.

One of the earliest approaches using GA for service composition was proposed in Canfora et al. (2005). This approach used time, cost, availability, and reliability as the QoS measures and a weighted ratio between these objectives as the basis for guiding the GA. The proposed method is compared to an integer programming approach on execution time, with the GA doing better for large problems while the integer programming approach worked better for small problems. In related research, these ideas were used to replan a process to ensure that QoS measures do not fall below those specified in service-level agreements (Canfora et al. 2005). Another early approach used generic QoS objectives and generic combining functions in a missed integer programming strategy (Alrifai and Risse 2009). In this case, the technique was evaluated on a synthetic process with varying number of tasks and services per task. Computation time and optimality were used to measure performance as the task count and service count were varied. Another early piece of research on service composition examined the use of workflow reduction techniques to estimate process-level measures of execution time, cost, encryption level, throughput, and availability (Jaeger et al. 2004). In a follow-up study, a GA was used to select services based on QoS measures of response time, cost, reputation, and availability (Jaeger and Muhl 2007). The GA was compared to 5 different heuristics using a pre-specified QoS ratio, and limitations in the GA noted.

A survey of prior research on service composition was presented in Strunk (2010). However, no new mechanisms were proposed. A related piece examines the selection of services utilizing QoS objectives of response time, cost, availability, and reputation (Ye et al. 2011). They use a GA-based approach on a synthetic process and compare the performance to an integer programming approach, utilizing execution time and overall fitness as the basis for comparison. Berbner et al. (2006) utilized three heuristics involving backtracking, swapping, and annealing. Using generic QoS measures on a synthetic process, they assessed the heuristics on computation time and approximation ratio. A heuristic-based approach that sought to identify local optimum solutions was proposed in Haddad et al. (2010). Using QoS objectives of response time, execution cost, availability, reputation, percent successful completion, and seven combinations of weights in the

presence and absence of risk, they assess overall performance based on the computation time and overall weighted score. Service composition has also been addressed in the selection of services in cloud-based environments (Asim et al. 2018; Baker et al. 2018; Baker et al. 2017; Baker et al. 2012; Baker et al. 2010; Karam et al. 2012; Kendrick et al. to appear). This stream of research is directed at the energy sector and examines the selection of services from single and multiple clouds. The problems are tackled from the cloud user and the cloud provider perspectives and address multiple objectives, including security, energy consumption, cost minimization, revenue maximization, and the like. A comparison of different approaches to service composition describing process structures supported, QoS objectives and their combination, solution technique, and application problems tackled appear in an appendix.

Clearly, many different approaches have been proposed for web service selection. Most of them have been tested on synthetic processes. The criteria, algorithms, and complexity of underlying business processes vary considerably. However, a more holistic perspective is needed that incorporates criteria that are relevant to practicing managers and is applicable to real-world business processes. This article develops a business-process-driven methodology for composing applications from available web services using multiple criteria decision support principles. Given the QoS measures of individual services, we develop techniques for deriving the end-to-end value of QoS measures for the composed application. An evolutionary heuristic approach using genetic algorithms is developed to effectively search through the alternatives and identify a Pareto-optimal multidimensional frontier that allows managers to make a trade-off among conflicting objectives. The method is applied to a real-world problem and results compared to another GA based multi-criteria approach proposed in the literature.

3 QUALITY OF SERVICE (QOS) METRICS FOR BUSINESS WEB SERVICES

Using non-functional objectives to select services for business applications requires that appropriate metrics are available for each service. Metrics should be precise, unambiguous, easily understood, reliable, robust, and easily quantified (Mills 1988). As SaaS becomes more widespread, QoS measures are increasingly incorporated into service-level agreements (SLA) (He et al. 2014). However, service-level metrics for reliability, performance, security, and cost need to be effectively combined to yield application-level measures. These application-level measures are a function of the characteristics of the services selected to support the business process and are shaped by the structure of the underlying business process. This section describes the service-level metrics selected for the four business objectives considered in this article, as adapted to the service context.

Software metrics typically measure characteristics of a specific software artifact, in this case, an individual service. The service could be deployed in-house or hosted by other parties, and hence the metric needs to be adjusted to include access to the service. Thus, performance will be composed of service time at the host, coupled with queue times at the service provider, and network traversal times in both directions. The use of additional protocols and security wrappers like SSL will cause additional performance degradation. Likewise, reliability measures need to encompass the service reliability at the host, coupled with the reliabilities of internal and external networks that connect the application to the service. In a similar vein, security metrics incorporate the security levels at the host, coupled with security provided by the network paths between the application and the host.

3.1 Service Reliability Metrics

Reliability of a service is a critical factor in the adoption of services (Kreger 2003), is considered an important QoS specification of a service (Ran 2003), and is often included in SLAs (Sahai et al. 2002). Drawing from the standard definition of software reliability, which is characterized as “the ability

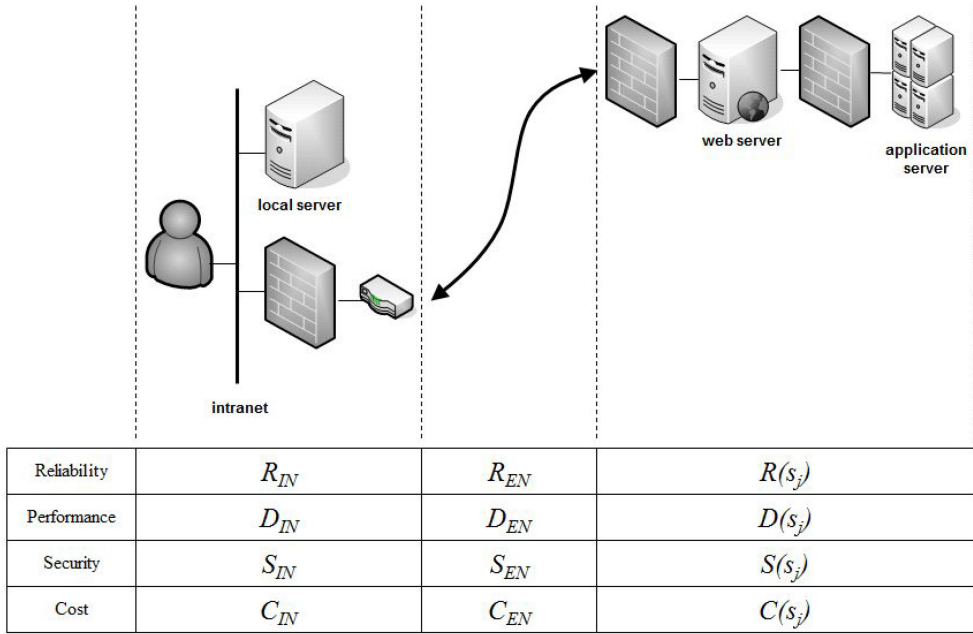


Fig. 1. Metrics for reliability, performance, security, and cost.

of a system or component to perform its required functions under stated conditions for a specified period of time” (Board 1991), service reliability can be viewed as the probability that the service will exhibit failure-free operation during a specified period. In the context of services to support business processes, reliability has been characterized as the “likelihood that the components will perform for its users on demand; it is a function of the failure rate” (Cardoso et al. 2002). For web services, an end-to-end measure of reliability is proposed in Vogels (2003), factoring in the network and message protocol reliabilities. Several measures of reliability have been adopted in the literature. The more common measures tend to rely on frequency of use, e.g., “the service fails once in every 10 invocations,” and are derived from the number of invocations and the number of failures. Other measures are based on elapsed time, e.g., “the service fails 15 times per month (or day/week/year),” and are derived from the failures observed over a period and averaged out over that period, regardless how often it is used during that period. Our research uses the former measure of reliability. We adapt this measure to address two contextual issues. First, the service may be executed at a remote location. Second, we allow a service to support multiple tasks, thereby increasing the reliability of the task. These two adaptations will apply for all other objectives, including performance, security, and cost. To illustrate the adaptations of the metrics as utilized in this article, we designate metrics for private networks, public networks, and service measures, as illustrated in Figure 1. The adapted metric for reliability, which considers redundant services, is illustrated in row 1 of Table 1.

3.2 Service Performance Metrics

Performance can be measured using a variety of metrics. Often a response time measure is employed. In the context of services to support business processes, this has been operationalized as response time for business tasks, which includes service times and delay (Cardoso et al. 2002), with delay composed of queueing and setup times. Taking into consideration the end-to-end nature of

Table 1. Adapted Service Metrics for Deployment

Criterion	Private network	Public network	Service measure	Adapted Metric	Metric with Redundant Service
Reliability	R_{IN}	R_{EN}	$R(s_j)$	$R_{IN} \times R_{EN} \times R(s_j)$	$1 - \prod_{k \in S} (1 - R(t_{ij}))$
Performance	D_{IN}	D_{EN}	$D(s_j)$	$D_{IN} + D_{EN} + D(s_j)$	$\min(D(t_{jk}) k \in S)$
Security	S_{IN}	S_{EN}	$S(s_j)$	$\min(S_{IN}, S_{EN}, S(s_j))$	$\max(S(t_{jk}) k \in S)$
Cost	C_{IN}	C_{EN}	$C(s_j)$	$C_{IN} + C_{EN} + C(s_j)$	$\min(C(t_{jk}) k \in S)$

service invocation, this research views performance as the total time needed for the service to support a given task or activity of a business process (Zo et al. 2010). We use this measure because it is most meaningful to managers making selection decisions, but it needs to be adapted for remote service invocation and the availability of redundant services. The end-to-end performance metric includes service processing times, communication times, and delays due to congestion, added security, and other protocols. For redundant services, the minimum value of service processing time is used, based on the notion that a faster service is a preferred service. Row 2 in Table 1 shows the adapted service performance metric.

3.3 Service Security Metrics

Metrics for software security are among the most challenging to derive. Several measures exist, including measures based on security risks, security defects, vulnerabilities, threats experienced, successful and thwarted attacks, among others (Zo et al. 2010). Metrics are reported as raw counts, densities, statistical analyses, time series analyses, scorecards, taxonomies, and hierarchies, among others. Though security is acknowledged as a critical QoS aspect of services (Ran 2003), usable measures for service security are limited. The few reported studies with concrete security measures for services employed percentage measures of compliance on multiple security dimensions and the use of radar graphs to generate an overall picture of service security (Serreli and Alexandris 2007). While these provide some insight, they are hard to combine across services that are coupled through various business-process patterns. In this research, security is measured as the percentage of transactions not compromised by security breaches at the service provider. The choice of a transaction-based measure, as opposed to an attack-based measure, is due to the notion that not all attacks are detectable, whereas compromised transactions are easily detected. This measure is most intuitive and meaningful to managers making service selection decisions, but it needs to be adapted for remote service invocation as well as availability of redundant services. The end-to-end security measure is shaped by security of the weakest link in the chain of resources that constitute the entire service invocation process. For redundant services, the maximum value is used, under the notion that a more secure service is a preferred service. Row 3 in Table 1 depicts the adapted service security metric.

3.4 Service Cost Metrics

At face value, cost is a straightforward metric. Combining service costs is also a simple process. However, this assumes that all services have similar pricing schemes. Clearly, that is not the case, and several pricing schemes are set forth by service providers, including outright purchase, lease for a specific duration, pay per use, and other more elaborate options as seen in the cloud computing environment (Harmon et al. 2009; Soni 2012). To effectively consider cost in the service selection model, we need a consistent basis for representing the cost of alternative services. For the purposes of this research, we normalize service cost by converting service price data to the

total cost of using the service for a fixed period, e.g., two years. Typically, the fixed period would be the life of the application or a fixed term based on the firm's accounting practices. Accordingly, the normalized service cost for a service s_j under various pricing policies could be of the following forms:

$$C(s_j) = PC(s_j) \quad (\text{Outright purchase}) \quad (1)$$

$$C(s_j) = FLC(s_j) + MLC(s_j) \times d(A_k) \quad (\text{Lease}) \quad (2)$$

$$C(s_j) = BTC(s_j) + TC(s_j) \times x(A_k) \quad (\text{Transaction-based cost}) \quad (3)$$

$$C(s_j) = BRC(s_j) + \sum_{l \in R} RC_l \times u_l \quad (\text{Resource-based cost}) \quad (4)$$

where

$PC(s_j)$ is the purchase cost of service s_j allocated to this application, with appropriate levels of depreciation and proportionate allocation if the service is used in multiple applications,

$FLC(s_j)$ is the fixed lease cost for service s_j ,

$MLC(s_j)$ is the monthly lease for service s_j ,

$d(A_k)$ is the anticipated life of the application A_k in months,

$BTC(s_j)$ is the base transaction cost of service s_j ,

$TC(s_j)$ is the unit transaction cost of service s_j ,

$x(A_k)$ is the anticipated number of transactions over the life of the application A_k ,

$BRC(s_j)$ is the base resource cost of service s_j ,

RC_l is the unit cost for use of resource l ,

u_l is the extent of usage of resource l over the life of the application A_k ,

R is the set of resources used by service s_j .

The usage-based costs need not be linear but could include step functions reflecting block pricing arrangements. When computing end-to-end cost of the service, the costs associated with internal networks, external network, security, and other procedures need to be allocated to the service, since infrastructure costs are shared across multiple applications. In the event of redundant services, the low-cost service will be the preferred service. Row 4 in Table 1 depicts the service cost metrics used in this research.

4 BUSINESS-PROCESS-DRIVEN APPLICATION COMPOSITION

A business process represents a collection of related business tasks that involve a number of actors and resources and collectively lead to an outcome that is of value to an organization or its customers (Dumas et al. 2013). There is a structure associated with these tasks that includes precedence, selection, looping, and other configurations. This structure is typically modeled using industry-standard modeling notations such as business-process modeling notations (BPMN).

4.1 Deriving Application-level Metrics

A business process may be a complex network employing many combining patterns. As a result, the overall reliability, performance, security, and cost of the application are not a simple combination of the individual reliability, performance, security, and cost of the services selected to support the application. Rather, a separate combining function based on the process structure needs to be developed for computing application-level values. A variety of different combining functions for deriving process metrics have been proposed (Benatallah et al. 2003; Canfora et al. 2005; Cardoso et al. 2002; Grønmo and Jaeger 2005; Hu 2003; Jaeger et al. 2005; Tsai et al. 2004). They offer different ways to combine the service-level metrics to form application-level metrics. However, they do not

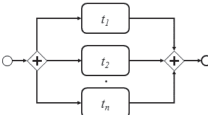
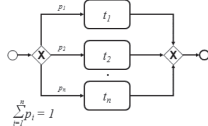
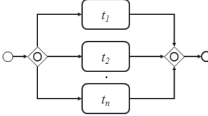
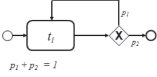
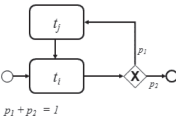
address all service criteria adopted in this article. Nor do they cover all process structures found in practice. A more comprehensive approach to computing application-level metrics is needed.

Drawing from research in business-process modeling, a number of patterns relating tasks to describe business processes can be identified. Business Process Modeling Notation (BPMN) includes basic patterns of task relationships that are commonly used in practice. Other approaches have identified larger sets of task relationship patterns (Russell et al. 2006; van der Aalst et al. 2003). These patterns add richness to the description of a business process, but have no impact on QoS measures for the process. Examples include event management patterns (e.g., initiation, termination, etc.), and timing patterns (synchronization, alarms, timed starts and stops). Others represent special cases of fundamental patterns, e.g., multi-way splits and parallel splits, or recursion and looping, and so on. When analyzing business-process patterns from a QoS perspective, it transpires that a small subset of patterns has an impact on the application QoS metrics. We identified six basic patterns that are consistently used to describe business processes in practice that are relevant for deriving application-level metrics. Accordingly, we focus on the six patterns. This represents a larger set than used in prior service composition research. We cataloged the number of patterns utilized in the research presented in the literature review. Five studies used just one pattern (viz. the sequence pattern), two studies used two patterns (sequence and parallel-AND), two studies used three patterns (sequence, parallel-AND, parallel-XOR), four studies used four patterns (sequence, parallel-AND, parallel-XOR, loop), and one study used five patterns (sequence, parallel-AND, parallel-XOR, parallel-OR, and loop). The patterns employed in this research are sequence, parallel-AND, parallel-XOR, parallel-OR, simple loop, and composite loop. The BPMN notation is used to illustrate the task relationship patterns. Table 2 shows the metrics for reliability, performance, security, and cost of each of the six patterns.

4.2 Composite Metrics for Reliability, Performance, Security, and Cost

The reliability for a process composed of services is derived from the reliability of individual services as well as the process structure. Functions for deriving composite reliability metrics are presented in Cardoso et al. (2002) and cover cases involving sequence, parallel-AND, parallel-XOR, and simple and composite loop patterns. These functions have been extended in Zo et al. (2012) to cover parallel-OR structures, availability of redundant services, as well as accommodate end-to-end application reliability measures, and are summarized in column 3 of Table 2. Likewise, metrics for performance are based on response time measures of individual services and the process structure. Drawing upon work in Cardoso et al. (2002) and extensions in Zo et al. (2010), composite performance measures are derived and depicted in column 4 of Table 2. Security metrics for the application are derived from the security measures of individual services as well as process structure. For security, the weakest-link principle is employed, whereby the overall security is shaped by the weakest level of security afforded in the process structure. We use the combining functions for composite security derived in Zo et al. (2010) and summarize them in column 5 of Table 2. At first glance, it would appear that deriving application level cost computation is a simple task. However, allocation of shared resource costs, coupled with different pricing models, make the cost computation harder. Prior research has adopted a unit transaction cost approach (Cardoso et al. 2002). However, given the range of pricing schemes used for services including outright purchase price, service lease, service usage, and resource-based pricing, a common basis is needed for realistic computation of application costs. Accordingly, a total cost of ownership approach is advocated in this research, where the total costs attributed to each of the selected services for the duration of the application life are used to arrive at the total cost of ownership of the application. Combining functions for composite cost measures for business processes that employ the six patterns are depicted in column 6 of Table 2.

Table 2. Combining Functions for the 4 Criteria for Various Business Patterns

Pattern	Representation	Reliability	Performance	Security	Cost
Sequence		$R(t_i) \times R(t_j)$	$D(t_i) + D(t_j)$	$\min(S(t_i), S(t_j))$	$C(t_i) + C(t_j)$
Parallel-AND		$\prod_{i=1}^n R(t_i)$	$\max(D(t_i); i \in 1, n)$	$\min(S(t_i); i \in 1, n)$	$\sum_{i=1}^n C(t_i)$
Parallel-XOR		$\sum_{i=1}^n p_i \cdot R(t_i)$	$\sum_{i=1}^n p_i \cdot D(t_i)$	$\sum_{i=1}^n p_i \cdot S(t_i)$	$\sum_{i=1}^n p_i \cdot C(t_i)$
Parallel-OR		$1 - \prod_{i=1}^n (1 - R(t_i))$	$\min(D(t_i); i \in 1, n)$	$\max(S(t_i); i \in 1, n)$	$\min(C(t_i); i \in 1, n)$
Simple Loop		$\frac{(1 - p_1) \cdot R(t_i)}{1 - p_1 \cdot R(t_i)}$	$\frac{D(t_i)}{1 - p_1}$	$S(t_i)$	$C(t_i)$
Composite Loop		$\frac{(1 - p_1) \cdot R(t_i)}{1 - p_1 \cdot R(t_i) \cdot R(t_j)}$	$\frac{D(t_i) - D(t_j)}{1 - p_1} - D(t_j)$	$\min(S(t_i), S(t_j))$	$C(t_i) + C(t_j)$

4.3 Alternative Approaches for Computing Application-level Composite Metrics

The combining functions described in Table 2 provide the basis for computing the overall reliability, performance, security, and cost metrics for an application composed of services. However, they specify the pattern for combining metrics for a pair of services only. This needs to be generalized to the entire application. One approach utilizes task agglomeration and is embodied by the Stochastic Workflow Reduction (SWR) algorithm (Kochut et al. 1999). The algorithm has been adapted and tailored to QoS metric computation for services in Cardoso (2002). The algorithm coalesces a pair of adjacent services into a larger service with reliability, performance, and cost for the combined service computed based on the combining functions outlined in Table 2. The process is repeated until the entire business process is reduced to one composite service. The reported complexity of the algorithm is $O(n^3 + nl)$ where n represents the number of tasks, and l the number of loops. The SWR algorithm is effective in computing the composite QoS measures for a given configuration of services. If any of the services in that configuration changes, then the algorithm must be run again. This is inefficient for a service selection problem, where many alternative services are to be considered to support individual tasks in a business process.

For the service selection problem addressed in this article, the business process is static, so a one-time formulation of the composite combining functions can be adopted. Various service selection alternatives can be evaluated by plugging into the combining function the appropriate values for reliability, performance, security, and cost. Since the number of possible combinations of services is expected to be large, this article uses a static combining function for the computation of composite metrics for reliability, performance, security, and cost of the composed application.

5 SERVICE SELECTION FOR COMPOSING BUSINESS APPLICATIONS

Some of the prior research addressing QoS metrics in the service selection problem has focused on individual QoS measures, including reliability (Zo et al. 2012), performance, and security (Zo et al. 2010). While these provide insight into how an individual QoS measure will affect the selection of services in an application, they do not generate a holistic perspective that managers need when selecting the most appropriate services. This research attempts to provide a holistic perspective that is applicable to business applications. Several strategies are available for service selection using multiple criteria. One approach involves setting minimum acceptable level on each objective and selecting the “best” feasible solution (Encarnación 1964). Weighted objective approaches have also been advocated, where the measures of each objective are normalized, and a combination of weights that reflects the designer’s preferences are used to select the “best” alternative (Hwang and Yoon 1981). Yet another approach involves the specification of an ideal solution, and the selection of the alternative that most closely mimics the ideal solution (Zeleny 1976). These approaches assume that the solution space is finite and all alternatives can be explored and evaluated. Clearly, this is not the case for the service selection problem. The use of multiple criteria introduces special challenges. Chief among these is the limitation that a weighted approach provides insights from a single manager’s perspective. A more general approach is needed, whereby a complete picture can be developed incorporating a range of perspectives based on any combination of the criteria. The service selection problem for composing business applications can be stated as follows:

Given a business process composed of tasks arranged in a specified structure, and a set of services to support these tasks, which services should be selected to provide the most preferred reliability, performance, security, and cost associated with the entire application?

We formulate the above problem as a multi-criteria optimization problem. Assume a business process P that consists of T tasks. For each task, assume that there are S_i viable services available. Let X represent the allocation of services to tasks. We define functions for cost, performance, security, and reliability as $f_1(x) = \text{Cost}(P|X)$, $f_2(x) = \text{Performance}(P|X)$, $f_3(x) = \text{Security}(P|X)$, and $f_4(x) = \text{Reliability}(P|X)$. Then the service selection problem can be stated as

$$\begin{aligned} \max_X \quad & f(x) = (f_1(x), f_2(x), f_3(x), f_4(x)) \\ \text{s.t.} \quad & \sum_{j \in S_i} x_{ij} = 1 \text{ for } i \in T \\ & x_{ij} = \{0, 1\} \text{ for } i \in T, j \in S_i. \end{aligned} \tag{5}$$

An additive weighted approach is often employed to combine the multiple criteria functions:

$$f(x) = \sum_{k=1}^4 w_k f_k(x)$$

where $\sum_{k=1}^4 w_k = 1$.

The criteria do not work in concert, i.e., the selected service will not have the highest reliability, perform the fastest, be the most secure, and cost the least. Instead, services are expected to fall

across a spectrum on each of these criteria. The selection problem is a class of problem commonly known as an NP-hard problem with multiple criteria. Since no one solution can be advanced as the optimal solution for this class of problems, an alternative approach is needed to arrive at a “preferred” solution.

5.1 Need for Heuristic Solution Approaches

With a reasonable number of candidate services available for each task in the business process, the size of the problem grows very quickly to levels where traditional search techniques become ineffective. For example, a business process consisting of 40 business tasks (which is quite realistic in practice), averaging 8 services to support each task, will have 8^{40} possible solutions, which is approximately 1.33×10^{36} . Traditional search techniques cannot address this in a reasonable time. If the business process grows larger, or the number of candidate services increase, then this becomes even more problematic.

Heuristic approaches represent an alternative solution in this case. Given a large number of alternatives to be explored, population-based heuristic solutions can be quite effective. These approaches typically require the ability to generate an initial set of solutions, a mechanism for evaluating these solutions, a process for generating a new set of solutions from the preferred solutions, and a stopping rule to end the process. A variety of population-based heuristic solution approaches exist, including genetic algorithms (Holland 1975), evolutionary programming (Fogel 1994), particle swarm optimization (Kennedy and Eberhart 1995), ant-colony optimization (Dorigo et al. 2006), artificial bee colonies (Karaboga and Basturk 2008), harmony search (Geem 2010), among others. The performance of these approaches is generally measured on two dimensions—the quality of the solution selected and the time or effort required to arrive at that solution. These measures are expected to be inversely related, in that more effort expended is likely to generate better solutions. The performance of the different approaches varies considerably and is dependent on the quality of evaluation function, the efficacy of generation of new solutions, the size of the solution space, nature of the solution space (particularly if there are pockets of local optima), and the stopping rule employed. We elected to use genetic algorithms, since it is a stable technique and has been successfully used for similar problems. Its convergence properties are well documented and its performance is generally acceptable, both in terms of the quality of solutions generated as well as the computational time and effort to arrive at the preferred solution.

5.2 Case Study: The Drop-Ship Business Process

To illustrate the effectiveness of our approach, a drop-ship case study that represents the real-life business process of product delivery by direct supply vendors (OASIS 2001) to online customers is employed. The process is moderately complex and spans 34 tasks, and is described in Figure 2. Assuming that an average of 6 candidate services is available for each task, the number of viable solutions is approximately 2.87×10^{26} . If an algorithm evaluated 1B candidate solutions each second, then the total search time for this problem would be approximately 9×10^9 years, making it infeasible for most optimization techniques. In an effort to make the problem more tractable, the process was decomposed into four sub-processes, involving customer identification, sales order processing, purchase order management, and shipping. These involved 4, 14, 6, and 10 tasks, respectively, with 24, 75, 33, and 57 candidate services available per sub-process. The number of possible solutions for each sub-process is 3.32×10^5 , 1.78×10^{26} , 1.29×10^9 , and 3.62×10^{17} , respectively. However, not all of these solutions are feasible, and a separate exploration of the solution space yielded 1.05×10^5 , 4.50×10^{17} , 8.20×10^6 , and 1.56×10^{12} feasible solutions, respectively. Despite the reduction of the original solution space, an exhaustive search for a solution is still not feasible, and a heuristic approach is necessary.

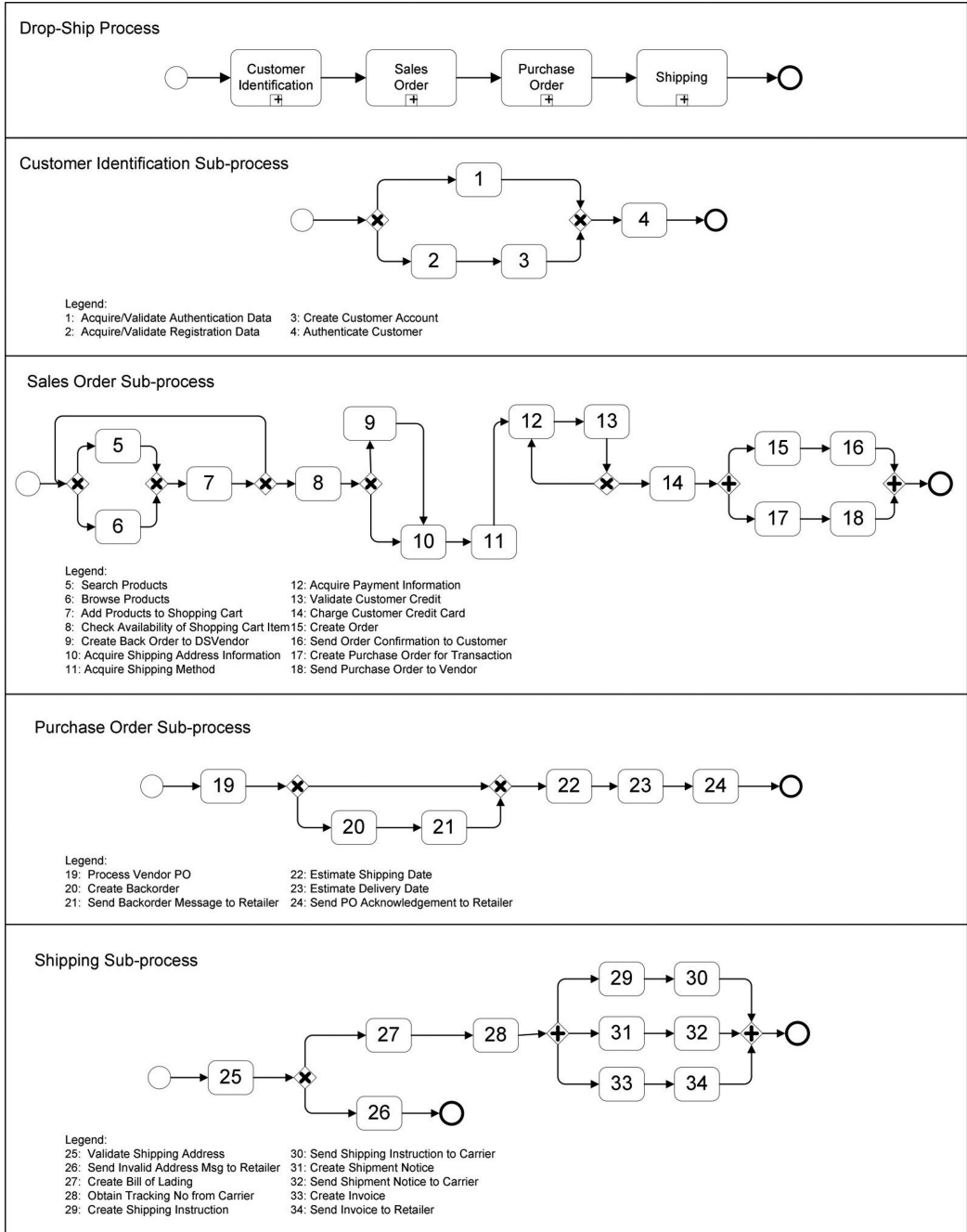


Fig. 2. Drop-ship business-process tasks and structure (BPMN notation).

For the purpose of testing our approach, we synthetically created services for each task. On average, 6 services were created for each task. In an effort to reflect the disparity of services available in the real world, services had varying levels of granularity, with some services being multi-functional in nature. QoS characteristics for the services were then generated, based on task characteristics as well as service granularity. Performance data was generated using transformed normal distributions. Security data was drawn from transformed inverted lognormal distributions. Reliability was based on transformed lognormal distributions. For the cost dimension, a variety of pricing models were employed, including outright purchase, leasing models, and usage-based models. All costs were converted to a single total cost of ownership for the duration of the application life—in this case, three years. Care was taken to ensure that the data exhibited realistic behavior not just within a criterion, but across all criteria. Thus, for example, the fastest service should not exhibit the highest level of reliability, the best security, and cost the least. An analysis of the service data indicated that the service characteristics were in line with expectation and did not include services that would be universally preferred or universally unattractive on all criteria. Frequency distributions were assembled for each QoS criteria and compared to the underlying distributions for consistency. In addition, correlational analysis was performed to determine if the criteria worked in concert. Finally, a dominance test was performed on the candidate services.

5.3 Solution Using Genetic Algorithms

Each solution in the service selection problem was represented as a chromosome composed of the concatenation of a binary string representation of the service selected for an individual task. 431 bits were needed to represent the selection of services for the 34 tasks in the process. The initial population size was set to 1K individuals. New solutions were generated using the upper 50% of the population. The crossover point was set to 0.8, and the mutation rate was fixed at 0.5. These selections led to the generation of better individuals in subsequent iterations, as well as reducing the search in local optima regions. A stopping rule of 1K iterations was employed, since the theoretical best value of the four criteria were not known in advance.

The evaluation or fitness function needs to incorporate end-to-end values for reliability, performance, security, and cost. The use of multiple criteria has been acknowledged as a concern in genetic algorithms, and several approaches have been proposed. An excellent summary is presented in Ghosh and Dehuri (2004). Some approaches rely on the dominance principle, eliminating solutions that are outperformed on all criteria. For the service selection problem, this results in the elimination of very few solutions. Other approaches examine individual criteria in isolation, which loses the holistic perspective needed to select the “best” solution. Two approaches consider the use of weights to capture the holistic perspective—the weighted sum approach (Ishibuchi and Murata 1998) and the weight-based genetic algorithm (Hajela and Lin 1992). The former uses random weights in the fitness function, while the latter employs user-provided weights. The use of random weights for generation of new solutions is acceptable, though less desirable for selection of a preferred solution from a manager’s perspective. With user-provided weights, the results are entirely dependent on the inputs selected by the individual. There is no opportunity for systematic exploration of the solution space. A different manager will prefer another set of weights, and the GA will have to be rerun. Even if the manager changes his or her weights, the entire process has to be repeated. A different approach that systematically explores all combinations of weights is employed in this research.

The fitness value for each criterion of reliability, performance, security, and cost are computed separately, based on the assignment of service to the task and using the combining functions outlined in Table 2. A weighted function combining the four criteria was assembled. This required that the scores on each criterion be normalized to a common scale, with scores for performance and

```

Assemble functions for Cost(DSP), Performance(DSP), Security(DSP), Reliability(DSP)
For each of 286 valid combination of criteria weights,  $W_i$ 
    Assemble fitness function  $f(\text{DSP})$  using Cost(DSP), Performance(DSP), Security(DSP), Reliability(DSP), and  $W_i$ 
    Generate initial population of 1000 solutions
    Evaluate  $f(\text{DSP})$  of each solution in initial population
    For each of 1000 iterations
        Generate new offspring using selection, crossover, and mutation functions
        Evaluate  $f(\text{DSP})$  of new offspring
        Assemble the new population by selecting the best parents and offspring
        Update the best and worst scores on each criterion for  $W_i$  if needed
    Loop for next iteration
    Record the best and worst scores on each criterion for  $W_i$ 
    Update the global best and worst scores on each criterion
Loop for next valid combination of criteria weights,  $W_i$ 

```

Fig. 3. Solution process employed in this research.

cost be reversed, since smaller values are preferred on these dimensions. The four-dimensional solution space was explored systematically, varying each criterion weight from 0 to 1 in steps of 0.1, with the constraint that the weights add up to 1. As a result, 286 distinct combinations of weights were identified. For each combination, an evaluation function was assembled, and the GA was run for each of these 286 combinations. The overall approach is summarized in Figure 3.

Application of this methodology to the drop-ship problem resulted in 286 different solutions, each of which represented the most preferred solution for a given combination of weights. For each of these solutions in addition to the service selected for each task, we also noted the normalized values of performance, reliability, security, and cost.

6 RESULTS

Our solution approach for the drop-ship case study was run on a workstation equipped with an Intel i5-760 Quad Core 2.6GHz processor with 4GB of memory running Windows 7 Enterprise Edition. Each run took between 47s and 53s. Rather than attempting to examine the 286 solutions in a four-dimensional space, a systematic exploration of these solutions was performed, providing greater insight into the effectiveness of the multi-criteria GA.

6.1 Analysis Based on Individual Criteria

The solutions were examined on cost, performance, security, and reliability separately. To facilitate comparison across dimensions, the data was normalized once again. An examination of the normalized scores using histograms and probability density functions provided little insight, and so we plotted the results relative to the weight of the criteria in the GA evaluation function. These results appear in Figure 4.

The graphs for the four criteria are in some respects similar. In all cases, the poorest scores were obtained when the criterion was assigned a zero weight. The best score for performance, security, and reliability was obtained when the criterion had a weight of one, but in the case of cost, this was attained with a weight of 0.9, which was 1.2% better than when cost was assigned a weight of 1. This illustrates the variance that can be expected from a heuristic technique like GA. Examination of the graphs reveals that good scores on each criterion can be accomplished at relatively low weights. In the case of cost, even with a zero weight, the high score attained was 0.73. When the weight was increased to 0.1, the high score jumped to 0.97 while the low score was 0.41. A similar trend was observed for the other criteria, albeit with some subtle differences. In the case of performance, a zero weight results in a low score of 0.00 and a high score of 0.40. While the low score is expected, the high score indicates that all-round solutions can be obtained with a reasonable level of performance even though performance is completely excluded from the objective function. As the weight for performance is increased to 0.1, the scores for performance

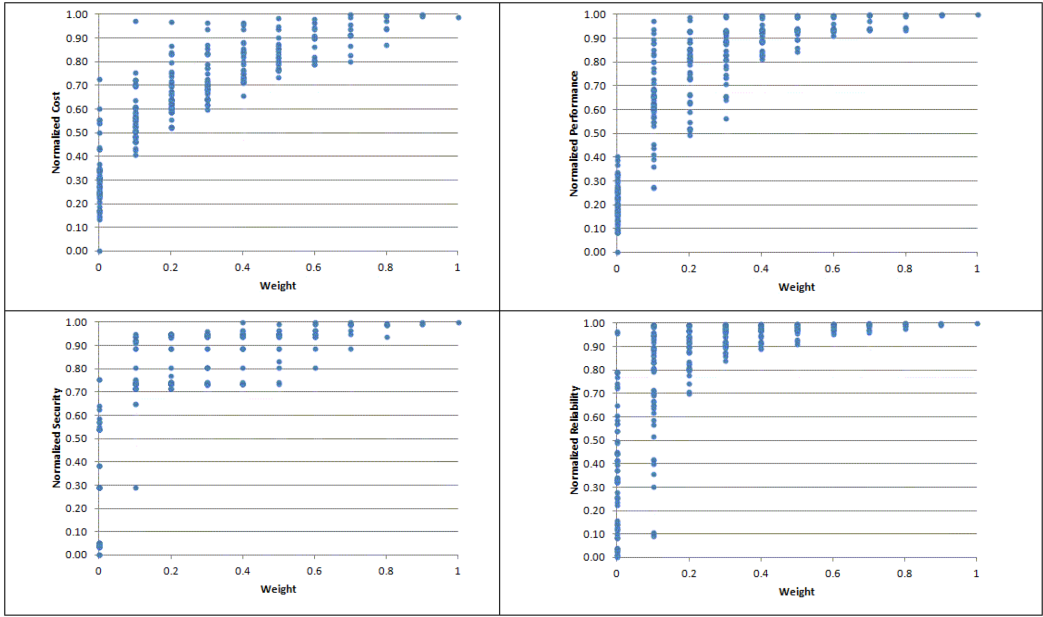


Fig. 4. Best solutions using single criteria.

improved with a range spanning 0.27 to 0.97. In the case of security, a similar pattern is observed. At a zero weight, the security scores range from 0.00 to 0.76. When the weight is increased to 0.1, the security score ranges from 0.29 to 0.95, which is a clear improvement. A single outlier distorted the picture, and with the outlier removed, the range is now 0.65 to 0.95, representing an impressive improvement. The case for reliability presented a slightly different picture. While the overall trend was the same, the dispersion noted was dramatically different. In this case, the normalized reliability scores ranged from 0.00 to 0.96 in the case of a zero weight for reliability. When the weight for reliability was moved to 0.1, the normalized reliability scores ranged from 0.09 to 0.99. Clearly, good scores on all dimensions can be obtained even when the weight assigned to the criteria is low. This analysis provides insight into the solution space to better equip the manager to make a trade-off between conflicting criteria.

6.2 Pair-wise Comparison of Criteria

Though the single criteria analysis provided some insight, it is not clear if a solution was good on multiple criteria. Likewise, no insights into the trade-off between the criteria are available, which is critical given the competing nature of the criteria. To facilitate this, a two-dimensional comparison of the criteria scores was performed. These results appear in Figure 5.

The results demonstrate the formation of an efficient frontier for many of the criteria pairs. An examination of the cost versus performance trade-off clearly illustrates this. Low-cost solutions exhibit poor performance, and improvements in performance are attained for solutions that are costlier. The convex frontier represents the best possible combination of cost and performance and illustrates the presence of no ideal solution. Designers and managers need to decide the importance of each criterion to select an appropriate solution. The efficient frontier is characterized by steep trade-offs, e.g., slight improvements on cost involve significant deterioration in performance and vice versa. The specific circumstances of the enterprise and its position within the industry will dictate the preferences on the cost-performance trade-off. The graph illustrates many dominated

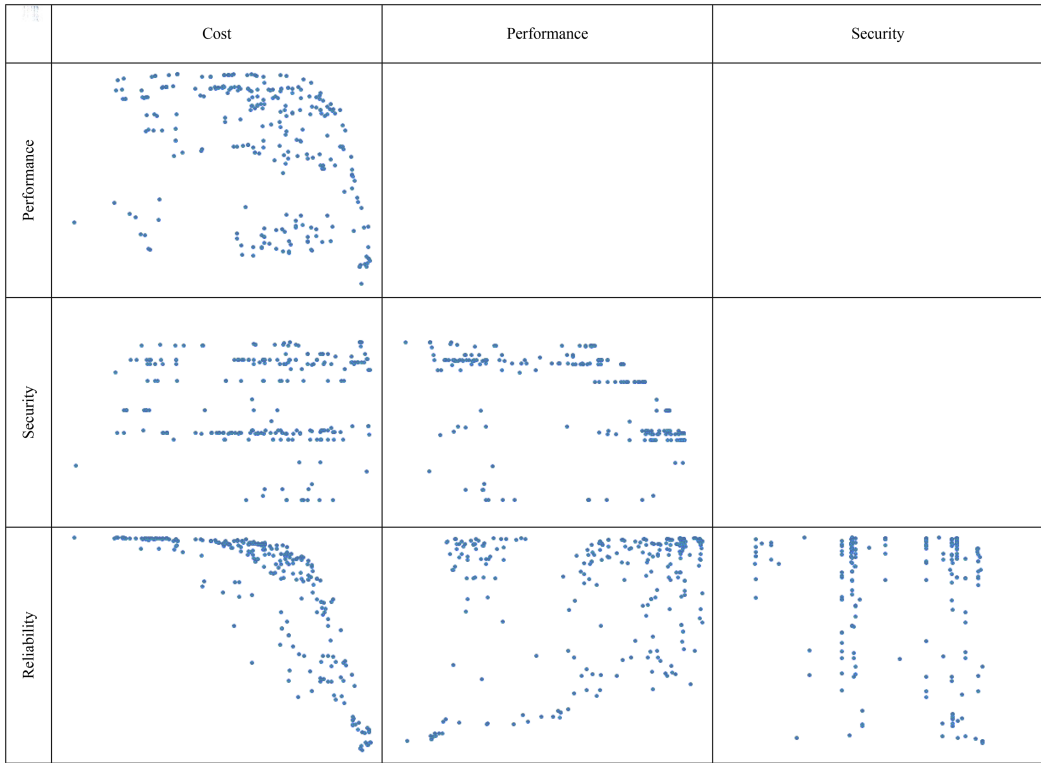


Fig. 5. Best solutions using criteria pairs.

solutions, but these are expected to have higher scores on reliability and security. In the case of cost and security, there are several low-cost solutions with high levels of security. While there is no ideal solution (lowest cost and highest security), there are several solutions that are relatively close. Cost and reliability represent a classic trade-off. While the typical efficient frontier is observed, there are two observations of note. First, the trade-off is asymmetric, with a gentle slope at the low-cost end, indicating that the deterioration in reliability involves non-trivial improvements in cost; and a steep slope at the high reliability end, indicating that marginal improvements in reliability are achieved only at steep cost trade-offs. Second, the dominated solutions lie close to the efficient frontier, suggesting that reliability and cost for the preferred solutions from each of the 286 combinations are closely aligned.

The trade-offs between security and performance are in line with expectations. As before, the range of security values observed is small, and the values exhibit interval clusters. Yet, the efficient frontier is consistent with the expectations. At the extremes of the frontier, a small increase in one objective is only obtained by a large deterioration on the other. The middle portions of the frontier exhibit a more natural trade-off pattern wherein a small improvement on objective entails a small deterioration on the other. The graph also illustrates a wide range of dominated solutions, which is expected. Reliability and performance trade-offs proved somewhat unexpected.

Several solutions had high levels of reliability and performance, suggesting an ideal solution. However, there are two additional observations of note. First, there is little to trade-off on either dimension, i.e., little room to improve on one dimension and a lot to lose on the other. Second, there is a large number of dominated solutions, with a significant drop-off on both dimensions

when compared to the preferred solutions. A closer examination of the frontier solutions revealed that they performed poorly on security and cost dimensions. Security and reliability also proved to be a little unexpected. In this case, ideal solutions should exhibit high levels of security as well as high levels of reliability. Several solutions were obtained close to the ideal solution. An examination of these solutions indicated that their cost and performance were considerably lower than the best values obtained. As before, the presence of near-ideal solutions on these two dimensions results in a large number of dominated solutions.

Unless there is complete independence between two criteria, visualizing the results in two dimensions will almost invariably yield an efficient frontier. This will enable managers to identify potentially good solutions on the two most important criteria, as well as establish a basis from trading off among criteria.

6.3 Holistic Assessment

While the pairwise comparison of criteria provides designers and managers guidance for trade-off, it is necessary to examine the results in a holistic manner to assess the true effectiveness of the approach. Visualizing data in four dimensions can be challenging. Multidimensional scaling can reduce the dimensions but is more useful for assessing the proximity of different solutions rather than their overall attractiveness. We employed several different approaches to help identify preferred solutions. We used the maximin technique, which involves identification of the minimum normalized score across all criteria and the selection of the highest such score. This yielded a solution that has lowest score of 0.74 on performance, for the combination of weights [0.3, 0.3, 0.3, 0.1], yielding normalized criteria scores of [0.77, 0.74, 0.89, 0.79] for cost, performance, security, and reliability, respectively, indicating that it is possible to find solutions that perform well on all criteria simultaneously. We then used average normalized scores, instead of the minimum score. In this case, the best average was 0.80, for the weight combination of [0.2, 0.2, 0.4, 0.2], and with scores of [0.64, 0.75, 0.89, 0.94] for cost, performance, security, and reliability, respectively. Unlike the first approach, which is strictly non-compensatory, this approach permits balancing low scores on one dimension by higher scores on other dimensions. A third approach involved the use of a 3-D plot, with a separate facet for the fourth dimension. Options considered for the facet of the fourth dimension included marker color, marker shading, and marker size. Varying the marker color provided the greatest degree of differentiation. We also experimented with using different options for the fourth dimension and finally elected to use performance, security, and reliability as the three visual dimensions and cost as the fourth dimension. These results are depicted in Figure 6.

The uppermost right vertex in the figure represents the most desirable solutions, as they have the highest reliability, security, and best performance. Solutions that are closer to the red end of the spectrum in this area represent the solutions with the lowest cost. Several solutions perform well on all criteria. The gradation of the color in the diagram indicates that some solutions are clearly superior on multiple dimensions to other solutions.

6.4 Assessment Using MCDM Techniques

Drawing from research in the multiple-criteria decision-making (MCDM) discipline, several techniques for selecting among alternatives that are described along common dimensions can be employed. Multi-attribute techniques that can be employed include a modified lexicographic or elimination by aspect approach. The manager specifies minimum acceptable performance on each criterion, and solutions that do not meet these requirements are eliminated. If no solutions remain, then the manager needs to relax each of these in turn, until the preferred solution emerges.

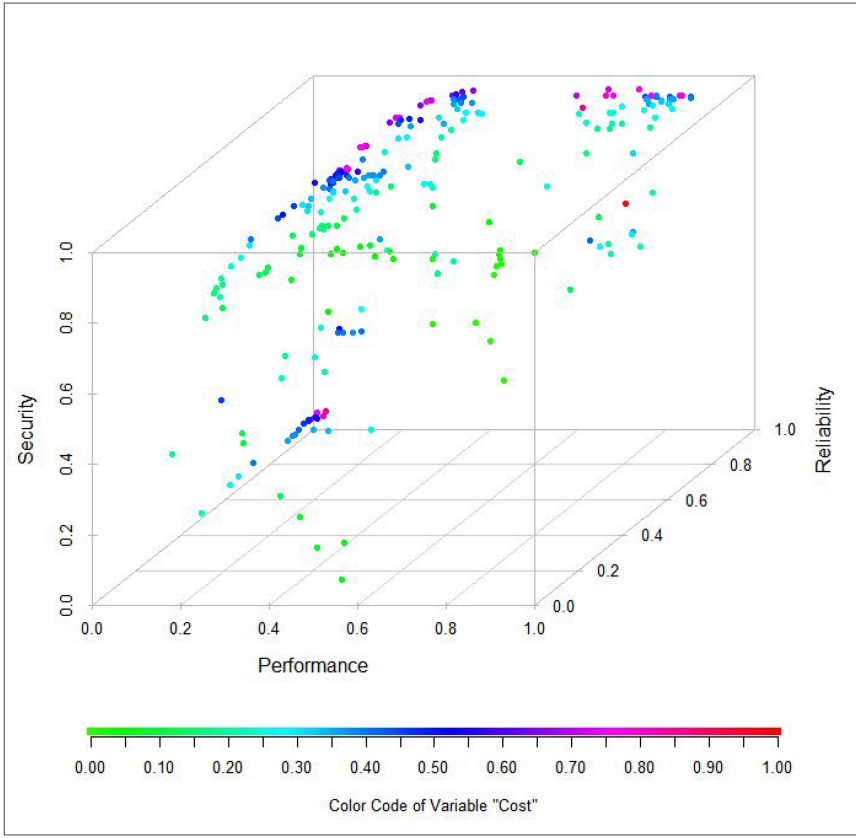


Fig. 6. 4-Dimensional representations of solutions.

A second strategy involves the use of the displaced ideal concept (Zeleny 1976). In this case, the manager specifies the ideal solution on the four dimensions provided, and an iterative process is adopted to trade-off competing criteria to arrive at the solution that best approximates this ideal. A variant to the trade-off is to employ a weighted distance from the ideal. In this case, a score can be computed for each solution given manager's weights as:

$$s_i = \sqrt{\sum_j w_j (1 - x_{ij})^2}$$

where x_{ij} represents the normalized value of alternative i on attribute j , and w_j represents the weight for attribute j . For equal weights, the preferred alternative has a score of 0.2103, representing the GA weight combination of [0.3, 0.3, 0.3, 0.1]. From the above cases, it becomes clear that more evenly distributed GA weights yield better all-round solutions. This is a significant finding that advises managers against giving extremely high importance to one criterion and low importance to others.

Dominance is another MCDM principle that can be employed. A quick check indicated that there were 74 dominated solutions. This left a total of 212 non-dominated solutions, which is a testament to the effectiveness of the GA process, indicating that the solutions generated are good in their own right. Selecting among these non-dominated solutions can now proceed using a modified MCDM solution. One approach that could work in this case was proposed in Jain and Dutta (1986). In this

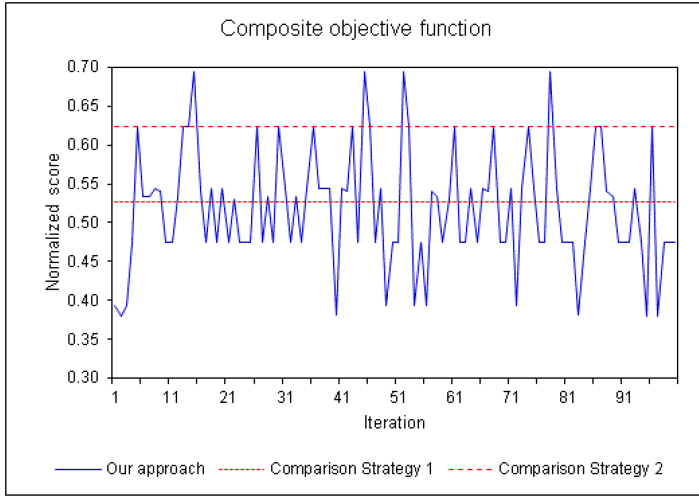


Fig. 7. Comparison with alternative GA-based approach.

case, the manager selects a promising solution and explores other solutions in the vicinity of that solution, specifying how the criteria can be traded off. The process continues until no trade-offs are deemed attractive relative to the current solution.

6.5 Comparison with Other Service Composition Approach

In an effort to benchmark against other GA-based service composition approaches, we ran a comparison with the approach outlined in Liu and Weng (2012). A true comparison can only be run if the service data are identical and the algorithms are reproduced faithfully. In this case, selection of the benchmark was dictated by the availability of service data. The lack of service data as well as parameter specification precluded comparison with other approaches. Prior research has either used execution time or normalized scores as a basis for comparison. Since differences in computing capability have a substantial effect on execution time, we elected to use normalized scores. Our selection was driven by the need to create a true comparison. Many of the other approaches to service composition utilize synthetic tasks with randomly generated services. This introduces a degree of non-comparability, since the services are not identical across the two approaches. Drawing from published service data in Liu and Weng (2012), we were able to fashion a strict equivalence between the two. The benchmark study used 6 tasks with 30 services. Using a predetermined set of weights, services were selected to maximize the overall normalized scores using task-specific and process-wide measures (labelled Comparison Strategy 1 and Comparison Strategy 2). We ran our algorithm 100 times to compare the results with the best overall scores recorded in the baseline paper, using different random number seeds. The results appear in Figure 7.

The best score obtained by our approach was 0.695, which represents an 11.6% improvement of the best score of 0.623 reported in the benchmark. Our approach was able to equal or outperform the best result for the benchmark in 52 out of 100 runs for the task-specific optimal strategy, and 19 out of 100 runs for the best solution in the process-wide optimal strategy. The results illustrate the inherent strengths and weaknesses of evolutionary heuristic techniques—while it is possible to attain convergence to a good solution in each run, the random nature of the technique can yield solutions of vastly differing quality. The comparison illustrates the capability of our approach to find high-quality solutions for a predetermined set of weights. Our approach also permits

exploration of a wide variety of combination of weights, making it more effective in supporting different managers, who may seek to emphasize different objectives under varied conditions.

7 MANAGERIAL IMPLICATIONS

This article presents a methodology to assemble a service-oriented application to support a business process. We found that a GA-based approach for selecting services for application composition provides an effective basis for managers to make a trade-off between competing criteria. The exploration of solutions generated by the GA-based approach provides a number of insights that are useful for developers and managers. At the outset, it illustrates that the approach proposed in this article can provide good and credible solutions to the complex problem of service selection when traditional optimization methods are inapplicable. The use of multiple criteria represents another useful contribution for managers. In this research, four QoS criteria were employed to guide the search. However, selecting among the alternative can prove challenging. Several techniques were outlined to make the decision more tractable.

The effectiveness of the GA proves to be a double-edged sword. It generates a high proportion of non-dominated solutions. Selecting among these can be difficult, though. This can be alleviated through the use of larger granularity weights. Employing a step size of 0.2 instead of 0.1 would entail 56 final solutions instead of 286, thereby making the selection simpler. We examined the subset generated by a coarser granularity approach and noted that the overall quality of the solutions using the minimum and the average normalized score was still the same, indicating no loss in quality, despite dropping to less than a quarter of the original set of solutions. Selecting an even more coarse set of weights is not recommended, since it will entail a very small set of final solutions to choose from.

This approach yields a static solution. If the emphasis among criteria needs to be re-evaluated, then it is a simple task to go back and perform the trade-offs once again, since the preferred solutions are already available. Should the set of available services or the tasks they support change, then the entire sequence of GAs needs to be run again. The use of the GA approach provides managers with the ability to identify good solutions, no matter what their underlying preferences are among the selected criteria. Since it generated a large percentage of non-dominated solutions, it provides managers with the confidence that they are selecting among the best options.

8 CONCLUSIONS

This research provides a holistic solution to the service selection problem when developing a business application composed of third-party services. The service composition problem is an NP-hard problem. Four distinct QoS criteria were employed. An application to a real-world drop-ship case study illustrated the approach. Given the complexity of the task, a genetic algorithm solution was adopted. End-to-end measures for each of these criteria were developed and applied in the fitness function for the GA. Preferences among the different criteria were systematically varied to cover the entire landscape of preferences. This approach allows managers to not only determine outcomes of their intrinsic preferences, but also permits them to examine the potential to trade off and improve the solution quality while still meeting their objectives. In addition, we ran a comparison with other service composition strategies and were able to match or better the performance in several cases.

This research makes several new contributions. First, it develops measures for security that are amenable to combining in a service composition context. Most service composition research includes security as a QoS objective but elect not to quantify it and include it in the composition problem. Second, in an effort to better reflect reality, it allows for multiple mechanisms for sourcing of services, including home-grown services as well as provided services. This has implications

for all QoS measures, since they may now involve private and public networks. Third, it permits multiple mechanisms for the provision of services, including in-house development, outright purchase, leasing, transaction and resource-based utilization. This flexibility lends additional realism but complicates the cost measures considerably. Fourth, given the nature of real-world services, this research assumes that a service could support multiple tasks. Multi-task support entails recalibration of the QoS measures, since fewer services can be used to support the process, and some degree of redundancy is now afforded in the process. Fifth, it employs a large real-world case, unlike prior research that either use small real-world cases or large synthetic cases. Sixth, it utilizes a multi-objective GA that systematically iterates over the entire user-preference landscape. Traditionally, service composition research selects a single combination of weights, or in some cases up to 7 combinations. Our approach examines 286 combinations and generates a vast majority of non-dominant solutions across these combinations, allowing managers to determine where they may choose to settle, rather than simply stating an up-front preference. Seventh, the use of multiple visualization techniques allows managers to hone in on specific objectives or trade off among multiple objectives. More importantly, it generates insights into the relationships between the various QoS objectives and lends clarity to whether the trade-off is a meaningful one. Finally, it provides a holistic solution to the application composition problem using web services. The approach is easily scaled for larger business processes. Likewise, it is flexible and extensible, should other QoS criteria need to be considered when composing applications from available web services.

APPENDIX

Comparison of Service Composition Approaches

Reference	Process Structures Supported	QoS Objectives	Combining Mechanism	Technique	Application	Assessment
Alrifai & Risse 2009	• Sequence	• Additive • Multiplicative • Minimum	• Additive • Multiplicative • Minimum	• Mixed IP	• Synthetic • 5–100 tasks • 50–2,000 WS per task	• Computation time • Optimality %
Amiri 2010	• Sequence • Parallel AND • Parallel XOR	• Response Time • Execution cost • Availability • Reputation • % successful	• None • User constraints	• GA	• Synthetic • 30,50,100 tasks • 30, 40, 50 WS	• Execution time • Compare with tabu search
Amiri et al. 2013	• Sequence • Parallel AND • Parallel XOR • Simple loop	• Response Time • Execution cost • Availability • Reputation • % successful	• None • User constraints	• GA	• Synthetic • 30,50,100 tasks • 30, 40, 50 WS	• Execution time • Compare with tabu search
Berbnner et al. 2006	• Sequence	• Additive • Multiplicative • Minimum	• Weighted normalized	• Backtracking • Heuristic swap • Heuristic anneal	• Synthetic • 3, 6, ..., 21 tasks • 40 WS per task	• Computation time • Approximation ratio
Bhiri et al. 2006	• Sequence • Parallel AND • Parallel XOR • Parallel OR	• Reliability	• None	• Transactional patterns	• Travel agent • 6 tasks	• None

Canfora et al. 2005	- Sequence - Parallel AND - Parallel XOR - Simple loop	- Response Time - Cost - Availability - Reliability - Custom attribute	Weighted ratio	GA	- Synthetic 8 tasks 5–25 WS per task	- Computation time - Compare with IP
El Haddad et al. 2010	- Sequence - Parallel AND - Parallel XOR	- Response Time - Execution cost - Availability - Reputation % successful	Weighted normalized (7 combinations)	Local optimum	- Synthetic 1–62 tasks 712–3602 WS	- Computation time - Weighted score
Elsayed et al. 2017	Sequence	- Response time - Cost - Reliability	Weighted normalized	GA with Q-learning	- Synthetic 9 tasks 500 WS per task	- Compare with GA - Compare with Q-learning
Jaeger et al. 2004	- Sequence - Parallel AND - Parallel OR - Parallel XOR - Simple loop	- Execution time - Cost - Encryption level - Throughput - Availability	None	Workflow reduction	None	None
Jaeger & Muhl 2007	Not specified	- Response time - Cost - Reputation - Availability	Additive weights	GA	- Synthetic 20 tasks 10 WS per task	Comparison with 5 heuristics
Kendrick et al. (to appear)	Flexible – input to output transform	- Cost - Energy efficiency	None	Agent-based search	- Synthetic 20,000 tasks	Compare with baseline
Liu et al. 2010	Sequence	- Response Time - Execution cost - Availability - Reputation	Weighted normalized	ACO-GA combination	- Synthetic 7 tasks 30 WS	Compare with ACO
Liu & Weng 2012	- Sequence - Parallel AND	- Response time - Cost - Reliability	- User constraints - Equal weights - Normalized	GA	- Real services 6 tasks 32 WS 3–8 per task	Normalized score
Ma & Zhang 2008	- Sequence - Parallel AND	5 unspecified objectives	Weights	Modified GA	- Synthetic 10, 25, 30 tasks 15 WS per task	Compare with traditional GA
Mardukhi et al. 2013	- Sequence - Parallel AND - Parallel XOR - Simple loop	- Response Time - Latency - Availability - Throughput - Reliability - Price	Normalized utility	GA with varying population size	- Real-world case 6 tasks 2,508 WS	Compare with mixed IP
Strunk 2010	- Sequence - Parallel AND - Parallel XOR - Simple loop	- Availability - Reliability - Execution time - Cost (price)	Normalized weights	None (survey only)	None	None

Su et al. 2007	- Sequence - Parallel AND	5 unspecified QoS properties	Normalized weights	GA with matrix representation	- Synthetic 10, 25, 30 tasks 15–50 WS per task	Compare with traditional GA
Tang & Ai 2010	- Sequence - Parallel AND - Parallel OR - Simple loop	- Response Time - Cost - Reputation - Reliability - Availability	Normalized weights	GA (3 variants)	- Synthetic 10–100 tasks 30 WS per task	Fitness value
Yang et al. 2010	Sequence	- Response Time - Cost	User constraints	ACO-GA combination	- Synthetic 10 tasks 35 WS per task	Compare with GA and immune algorithm
Ye et al. 2011	- Sequence - Parallel AND - Parallel XOR - Simple loop	- Response Time - Cost - Availability - Reputation	- User constraints - Equal weights	GA	- Synthetic - Up to 100 tasks 30 WS per task	Compare with IP
This research	- Sequence - Parallel AND - Parallel OR - Parallel XOR - Simple loop - Composite loop	- Response Time - Cost - Security - Reliability	- Weighted normalized 286 combinations	Multi-objective GA	- Real-world case 34 tasks 189 WS	- Normalized score - Dominance - Compare with GA

REFERENCES

- Y. Achbany, I. J. Jureta, S. Faulkner, and F. Fouss. 2008. Continually learning optimal allocations of services to tasks. *IEEE Trans. Serv. Comput.* 1, 3 (2008), 141–154.
- W. Ahmed, Y. W. Wu, and W. M. Zheng. 2015. Response time based optimal web service selection. *IEEE Trans. Parallel Distrib. Syst.* 26, 2 (2015), 551–561.
- A. O. Akingbesote, M. O. Adigun, J. B. Oladosu, and E. Jembere. 2013. A quality of service aware multi-level strategy for selection of optimal web service. In *Proceedings of the International Conference on Adaptive Science and Technology (ICAST'13)*. 1–5.
- M. Alrifai and T. Risse. 2009. Combining global optimization with local selection for efficient QoS-aware service composition. In *Proceedings of the 18th International Conference on World Wide Web*. 881–890.
- M. A. Amiri, V. Derhami, and M. Ghasemzadeh. 2013. QoS-based web service composition based on genetic algorithm. *J. AI Data Mining* 1, 2 (2013), 63–73.
- M. A. Amiri and H. Serajzadeh. 2010. QoS aware web service composition based on genetic algorithm. In *Proceedings of the 5th International Symposium on Telecommunications*. 502–507.
- M. Asim, A. Yautsiukhin, A. D. Brucker, T. Baker, Q. Shi, and B. Lempereur. 2018. Security policy monitoring of BPMN-based service compositions. *J. Softw.: Evol. Proc.* 30, 3 (2018), 1–17.
- T. Baker, B. Aldawsari, M. Asim, H. Tawfik, Z. Maamar, and R. Buyya. 2018. Cloud-SEnergy: A bin-packing based multi-cloud service broker for energy efficient composition and execution of data-intensive applications. *Sustain. Comput.: Inform. Syst.* 19 (2018), 242–252.
- T. Baker, M. Asim, H. Tawfik, B. Aldawsari, and R. Buyya. 2017. An energy-aware service composition algorithm for multiple cloud-based IoT applications. *J. Netw. Comput. Appl.* 89, 96–108.
- T. Baker, A. Taleb-bendiab, M. Randles, and A. Hussien. 2012. Understanding elasticity of cloud services compositions. In *Proceedings of the IEEE 5th International Conference on Utility and Cloud Computing*.
- T. Baker, A. Taleb-Bendiab, M. Randles, and Y. Karam. 2010. Support for adaptive cloud-based applications via intention modelling. In *Proceedings of the 3rd International Symposium on Web Services Zayed University*. 235–243.
- B. Benatallah, Q. Z. Sheng, and M. Dumas. 2003. The self-serv environment for web services composition. *IEEE Internet Comput.* 7, 1 (2003), 40–48.
- A. Benlian, M. Koufaris, and T. Hess. 2011. Service quality in software-as-a-service: Developing the SaaS-Qual measure and examining its role in usage continuance. *J. Manag. Inform. Syst.* 28, 3 (2011), 85–126.

- R. Berbner, M. Spahn, N. Repp, O. Heckmann, and R. Steinmetz. 2006. Heuristics for QoS-aware web service composition. In *Proceedings of the IEEE International Conference on Web Services (ICWS'06)*.
- S. Bhiri, W. Gaaloul, and C. Godart. 2006. Discovering and improving recovery mechanisms of composite web services. In *Proceedings of the International Conference on Web Services (ICWS'06)*. 99–110.
- I. S. Board. 1991. *IEEE standard glossary of software engineering terminology*. IEEE, New York, NY.
- G. Canfora, M. Di Penta, R. Esposito, and M. L. Villani. 2005. QoS-aware replanning of composite web services. In *Proceedings of the IEEE International Conference on Web Services (ICWS'05)*. 121–129.
- G. Canfora, M. D. Penta, R. Esposito, and M. L. Villani. 2005. An approach for QoS-aware service composition based on genetic algorithms. In *Proceedings of the Conference on Genetic and Evolutionary Computation (GECCO'05)*. ACM Press, 1069–1075.
- G. Canfora, M. D. Penta, R. Esposito, and M. L. Villani. 2005. QoS-aware replanning of composite web services. In *Proceedings of the IEEE International Conference on Web Services (ICWS'05)*. 121–129.
- J. Cardoso. 2002. *Stochastic Workflow Reduction Algorithm*. Technical Report. LSDIS Lab, Department of Computer Science, University of Georgia, Athens, GA.
- J. Cardoso, J. Miller, A. Sheth, and J. Arnold. 2002. *Modeling Quality of Services for Workflows and Web Services Processes*. Technical Report. LSDIS Lab, Department of Computer Science, University of Georgia, Athens, GA.
- J. Cardoso, A. P. Sheth, and K. J. Kochut. 2002. *Implementing QoS Management for Workflow Systems*. Technical Report. LSDIS Lab, Department of Computer Science, University of Georgia, Athens, GA.
- J. Cardoso, A. P. Sheth, J. A. Miller, J. Arnold, and K. J. Kochut. 2004. Quality of service for workflows and web service processes. *Web Seman.: Sci., Serv. Agents World Wide Web 1*, 3 (2004), 281–308.
- M. Carman, L. Serafini, and P. Traverso. 2003. Web service composition as planning. In *Proceedings of the Workshop on Planning for Web Services*. 1636–1642.
- Y. Chen, J. W. Huang, C. Lin, and J. Hu. 2015. A partial selection methodology for efficient QoS-aware service composition. *IEEE Trans. Serv. Comput.* 8, 3 (2015), 384–397.
- J. Choi, D. L. Nazareth, and H. K. Jain. 2010. Implementing service-oriented architecture in organizations. *J. Manag. Inform. Syst.* 26, 4 (2010), 253–286.
- J. Clabby. 2002. *Web Services Explained, Solutions and Applications for the Real World*. Prentice Hall PTR, Upper Saddle River, NJ.
- H. J. Dai, H. Qu, J. H. Zhao, W. H. Dong, and W. J. Xie. 2014. A distributed optimal scheme based on local QoS for web service composition. *China Commun.* 11, 13 (2014), 142–147.
- J. C. Delano, H. K. Jain, and A. P. Sinha. 2018. System design through the exploration of contemporary web services. *ACM Trans. Manag. Inform. Syst.* 9, 3 (2018), Article 13.
- M. Dorigo, M. Birattari, and T. Stutzle. 2006. Ant colony optimization—Artificial ants as a computational intelligence technique. *IEEE Comput. Intell. Mag.* 1, 4 (2006), 28–39.
- M. Dumas, M. La Rosa, J. Mendling, and H. Reijers. 2013. *Fundamentals of Business Process Management*. Springer-Verlag, Berlin.
- D. H. Elsayed, E. S. Nasr, Alaa El Din M. El Ghazali, and M. H. Gheith. 2017. A new hybrid approach using genetic algorithm and Q-learning for QoS-aware web service composition. In *Proceedings of the International Conference on Advanced Intelligent Systems and Informatics (AISIT'17)*, S. K. Hassanien A. Gaber T. Tolba M. (Eds.). 537–546.
- J. J. Encarnación. 1964. A note on lexicographical preferences. *Econometrica* 32, 1–2, 215–217.
- D. B. Fogel. 1994. An introduction to simulated evolutionary optimization. *IEEE Trans. Neural Netw.* 5, 1 (1994), 3–14.
- Z. W. Geem. 2010. State-of-the-art in the structure of harmony search algorithm. In *Recent Advances in Harmony Search Algorithm*, Z.W. Geem (Ed.). Springer-Verlag, Berlin, 1–10.
- A. Ghosh and S. Dehuri. 2004. Evolutionary algorithms for multi-criterion optimization: A survey. *Int. J. Comput. & Inform. Sci.* 2, 1 (2004), 38–57.
- R. Grønmo and M. C. Jaeger. 2005. Model-driven methodology for building QoS-optimised web service compositions. In *Proceedings of the 5th IFIP International Conference on Distributed Applications and Interoperable Systems (DAIS'05)*. 68–82.
- R. Grønmo and M. C. Jaeger. 2005. Model-driven semantic web service composition. In *Proceedings of the 12th Asia-Pacific Software Engineering Conference (APSEC'05)*.
- R. Grønmo, D. Skogan, I. Solheim, and J. Oldevik. 2004. Model-driven web service development. *Int. J. Web Serv. Res.* 1, 4 (2004), 1–13.
- J. E. Haddad, M. Manouvrier, and M. Rukoz. 2010. TQoS: Transactional and QoS-aware selection algorithm for automatic web service composition. *IEEE Trans. Serv. Comput.* 3, 1 (2010), 73–85.
- P. Hajela and C. Y. Lin. 1992. Genetic search strategies in multicriterion optimal-design. *Struct. Optim.* 4, 2 (1992), 99–107.
- R. Harmon, H. Demirkan, B. Hefley, and N. Auseklis. 2009. Pricing strategies for information technology services: A value-based approach. In *Proceedings of the 42nd Hawaii International Conference on System Sciences (HICSS'09)*. 1–10.

- K. Hashmi, Z. Malik, E. Najmi, A. Alhosban, and B. Medjahed. 2016. A web service negotiation management and QoS dependency modeling framework. *ACM Trans. Manag. Inform. Syst.* 7, 2 (2016), Article 5.
- O. Hatzi, D. Vrakas, M. Nikolaidou, N. Bassiliades, D. Anagnostopoulos, and I. Vlahavas. 2012. An integrated approach to automated semantic web service composition through planning. *IEEE Trans. Serv. Comput.* 5, 3 (2012), 319–332.
- Q. He, J. Yan, H. Jin, and Y. Yang. 2014. Quality-aware service selection for service-based systems based on iterative multi-attribute combinatorial auction. *IEEE Trans. Softw. Eng.* 40, 2 (2014), 192–215.
- J. H. Holland. 1975. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. University of Michigan Press, Ann Arbor, MI.
- M. Hu. 2003. Web services composition, partition, and quality of service in distributed system integration and re-engineering. In *Proceedings of the XML Conference and Exposition*.
- Y. Huaizhou and W. Fan. 2014. A conflict-free service selection algorithm in the situation of complex service dependency relationships. In *Proceedings of the 5th International Conference on Intelligent Systems Design and Engineering Applications (ISDEA'14)*. 9–13.
- C.-L. Hwang and K. Yoon. 1981. *Multiple Attribute Decision Making: Methods and Applications A State-of-the-Art Survey*. Springer-Verlag, Berlin.
- H. Ishibuchi and T. Murata. 1998. A multi-objective genetic local search algorithm and its application to flowshop scheduling. *IEEE Trans. Syst. Man Cyber. Part C-Appl. Rev.* 28, 3 (1998), 392–403.
- M. C. Jaeger and G. Muhl. 2007. QoS-based selection of services: The implementation of a genetic algorithm. In *Proceedings of the 15th ITG/GI Symposium on Communication in Distributed Systems*. 1–12.
- M. C. Jaeger, G. Muhl, and S. Golze. 2005. QoS-aware composition of web services: A look at selection algorithms. In *Proceedings of the IEEE International Conference on Web Services (ICWS'05)*. 807–808.
- M. C. Jaeger, G. Rojec-Goldmann, and G. Muhl. 2004. QoS aggregation for web service composition using workflow patterns. In *Proceedings of the 8th IEEE International Enterprise Distributed Object Computing*.
- H. K. Jain and A. Dutta. 1986. Distributed computer system design: A multicriteria decision-making methodology. *Dec. Sci.* 17, 4 (1986), 437–453.
- W. Jiang, S. L. Hu, and Z. Y. Liu. 2014. Top k query for QoS-aware automatic service composition. *IEEE Trans. Serv. Comput.* 7, 4 (2014), 681–695.
- N. Joachim. 2011. A literature review of research on serviceoriented architectures (SOA): Characteristics, adoption determinants, governance mechanisms, and business impact. In *Proceedings of the 17th Americas Conference on Information Systems (AMCIS'11)*.
- D. Karaboga and B. Basturk. 2008. On the performance of artificial bee colony (ABC) algorithm. *App. Soft Comput.* 8, 1 (2008), 687–697.
- Y. Karam, T. Baker, and A. Taleb-bendiab. 2012. Security support for intention driven elastic cloud computing. In *Proceedings of the 6th UKSim/AMSS European Symposium on Computer Modeling and Simulation*.
- P. Kendrick, T. Baker, Z. Maamar, A. Hussain, R. Buyya, and D. Al-Jumeily. to appear. An efficient multi-cloud service composition using a distributed multiagent-based, memory-driven approach. *IEEE Trans. Sustain. Comput.*
- J. Kennedy and R. Eberhart. 1995. Particle swarm optimization. In *Proceedings of the IEEE International Conference on Neural Networks*. 1942–1948.
- K. Kochut, A. Sheth, and J. A. Miller. 1999. Optimizing workflow. *Compon. Strat.* 1 (1999), 45–57.
- H. Kreger. 2003. Fulfilling the web services promise. *Commun. ACM* 46, 6 (2003), 29.
- H. Liu, F. Zhong, B. Ouyang, and J. Wu. 2010. An approach for QoS-aware web service composition based on improved genetic algorithm. In *Proceedings of the International Conference on Web Information Systems and Mining*. 123–128.
- S.-C. Liu and S.-S. Weng. 2012. Applying genetic algorithm to select web services based on workflow quality of service. *J. Electron. Commer. Res.* 13, 2 (2012), 157–172.
- Y. Ma and C. Zhang. 2008. Quick convergence of genetic algorithm for QoS-driven web service selection. *Comput. Netw.* 52, 5 (2008), 1093–1104.
- N. B. Mabrouk, S. Beauche, E. Kuznetsova, N. Georgantas, and V. Issarny. 2009. QoS-aware service composition in dynamic service oriented environments. In *Proceedings of the 10th International Middleware Conference (Middleware'09)*. 123–142.
- F. Mardukhi, N. Nematbakhsh, K. Zamanifar, and A. Barati. 2013. QoS decomposition for service composition using genetic algorithm. *Appl. Soft Comput.* 13, 7 (2013), 3409–3421.
- B. Medjahed, A. Bouguettaya, and A. K. Elmagarmid. 2003. Composing web services on the semantic web. *VLDB J.* 12, 4 (2003), 333–351.
- E. E. Mills. 1988. *Software Metrics - SEI Curriculum Module SEI-CM-12-1.1*. Software Engineering Institute, Pittsburgh, PA.
- OASIS. 2004. WS-Reliability 1.1. In *Web Services Reliable Messaging TC, OASIS Standard, Burlington, WA*.
- UN/CEFACT and OASIS. 2001. Business process analysis worksheets & guidelines v1.0: Procedures for developing business processes in ebXML. Business Process Team, OASIS, Burlington, WA.

- M. P. Papazoglou and D. Georgakopoulos. 2003. Service-oriented computing: Introduction. *Commun. ACM* 46, 10 (2003), 24–28.
- R. Ramacher and L. Monch. 2015. Service selection with runtime aspects: A hierarchical approach. *IEEE Trans. Serv. Comput.* 8, 3 (2015), 481–493.
- A. Ramirez, J. A. Parejo, J. R. Romero, S. Segura, and A. Ruiz-Cortes. 2017. Evolutionary composition of QoS-aware web services: A many-objective perspective. *Exp. Syst. Appl.* 72 (2017), 357–370.
- S. Ran. 2003. A model for web services discovery with QoS. *ACM SIGecom Exch.* 4 (2003), 1–10.
- P. Rodriguez-Mier, M. Mucientes, and M. Lama. 2017. Hybrid optimization algorithm for large-scale QoS-aware service composition. *IEEE Trans. Serv. Comput.* 10, 4 (2017), 547–559.
- N. Russell, A. H. M. Ter Hofstede, W. M. P. Van Der Aalst, and N. Mulyar. 2006. Workflow control-flow patterns: A revised view. Technical Report BPM-06-22, BPM Center, Eindhoven, Netherlands.
- A. Sahai, A. Durante, and V. Machiraju. 2002. *Towards Automated SLA Management for Web Services*. Technical Report HPL-2001-310. HP Laboratories, Palo Alto, CA.
- M. A. Serhani, R. Dssouli, A. Hafid, and H. Sahraoui. 2005. A QoS broker based architecture for efficient web services selection. In *Proceedings of the IEEE International Conference on Web Services (ICWS'05)*. 113–120.
- E. Serrelis and N. Alexandris. 2007. An empirical model for quantifying security based on services. In *Proceedings of the International Multi-Conference on Computing in the Global Information Technology (ICCGI'07)*. 30.
- P. Shahrokh and F. Safi-esfahani. 2016. QoS-based web service composition applying an improved genetic algorithm IGA method. *Int. J. Ent. Inform. Syst.* 12, 3 (2016), 60–77.
- M. Soni. 2012. Cloud computing and chargeback models. *Cloudbook* 3, 1 (2012), 42–56.
- A. Strunk. 2010. QoS-aware service composition: A survey. In *Proceedings of the 8th IEEE European Conference on Web Services*. 67–74.
- S. Su, C. Zhang, and J. Chen. 2007. An improved genetic algorithm for web services selection. In *Proceedings of the Distributed Applications and Interoperable Systems Conference (DAIS'17)*, R. K. Indulska J. (Ed.). 284–295.
- B. Suleiman, C. E. Da Silva, and S. Sakr. 2011. One size does not fit all: A group-based service selection for web-based business processes. In *Proceedings of the IEEE Workshops of International Conference on Advanced Information Networking and Applications (WAINA'11)*. 253–260.
- M. Tang and L. Ai. 2010. A hybrid genetic algorithm for the optimal constrained web service selection problem in web service composition. In *Proceedings of the IEEE Congress on Evolutionary Computation*. 1–8.
- M. Ter Beek, A. Buchiarone, and S. Gnesi. 2007. Web service composition approaches: From industrial standards to formal methods. In *Proceedings of the 2nd International Conference on Internet and Web Applications and Services (ICIW'07)*. 15–20.
- W. T. Tsai, D. Zhang, Y. Chen, H. Huang, R. Paul, and N. Liao. 2004. A software reliability model for web services. In *Proceedings of the 8th IASTED International Conference on Software Engineering and Applications*. 144–149.
- W. M. P. Van Der Aalst, A. H. M. Ter Hofstede, B. Kiepuszewski, and A. P. Barros. 2003. Workflow patterns. *Distrib. Parallel Data.* 14, 1 (2003), 5–51.
- W. Vogels. 2003. Web services are not distributed objects. *IEEE Internet Comput.* 7, 6 (2003), 59–66.
- Z. Yang, C. Shang, Q. Liu, and C. Zhao. 2010. A dynamic web services composition algorithm based on the combination of ant colony algorithm and genetic algorithm. *J. Comput. Inform. Syst.* 6, 8 (2010), 2617–2622.
- Z. Ye, X. Zhou, and A. Bouguettaya. 2011. Genetic algorithm based QoS-aware service compositions in cloud computing. In *Proceedings of the International Conference on Database Systems for Advanced Applications (DASFAA'11)*. 321–334.
- M. Zeleny. 1976. The theory of the displaced ideal. In *Multiple Criteria Decision Making Kyoto 1975*, M. Zeleny (Ed.) Springer-Verlag, Berlin, 153–206.
- L. P. Zhao, Y. Z. Ren, M. C. Li, and K. Sakurai. 2012. Flexible service selection with user-specific QoS support in service-oriented architecture. *J. Netw. Comput. Appl.* 35, 3 (2012), 962–973.
- H. Zo, D. L. Nazareth, and H. K. Jain. 2010. Security and performance in service-oriented applications: Trading off competing objectives. *Dec. Supp. Syst.* 50, 1 (2010), 336–346.
- H. Zo, D. L. Nazareth, and H. K. Jain. 2012. End-to-end reliability of service oriented applications. *Inform. Syst. Front.* 14, 5 (2012), 971–986.
- G. B. Zou, Q. Lu, Y. X. Chen, R. Y. Huang, Y. Xu, and Y. Xiang. 2014. QoS-aware dynamic composition of web services using numerical temporal planning. *IEEE Trans. Serv. Comput.* 7, 1 (2014), 18–31.

Received October 2018; revised June 2019; accepted August 2019